

Full Stack Development with MERN

DOCSPOT (Medical Appointment Management System)

1.Introduction

1.1 Project Title

Project Title: Doctor Booking– Medical Appointment Management System (MERN)

1.2 Member

Member : Karthikeyan S

◆ **Role:** Full Stack Developer

Responsibilities:

- ◆ Developed the complete frontend using React for user, doctor, and admin modules.
- ◆ Created forms, dashboards, tables, and UI components for smooth application flow.
- ◆ Integrated the frontend with backend APIs using Axios.
- ◆ Implemented user authentication and role-based access using JWT.
- ◆ Designed MongoDB schemas for users, doctors, and appointments.
- ◆ Built backend APIs using Node.js and Express for all major features.
- ◆ Added document upload functionality for appointments.
- ◆ Handled state management, validation, and error handling in the UI.
- ◆ Tested all modules manually to ensure proper functionality.
- ◆ Prepared project documentation and managed overall project structure.

2. Project Overview

2.1 Purpose

CareSync Portal is a web-based doctor appointment management system built using the MERN stack (MongoDB, Express.js, React, Node.js).

The purpose of the system is to:

- Allow **patients** to register, log in, browse available doctors, upload relevant medical documents, and request appointments.
- Allow **doctors** to view incoming appointment requests, approve them, and download attached documents.
- Allow **administrators** to manage users, doctor profiles, and monitor all appointments centrally.

Overall, the project aims to **reduce manual appointment handling**, avoid long phone calls, and bring transparency to the booking process. It centralizes patient–doctor interaction and makes appointment management more structured and efficient.

2.2 Project Goals

- Provide an intuitive **patient dashboard** for viewing and managing appointments.
- Provide a clear **doctor dashboard** to manage appointment requests with patient details and attached documents.
- Implement an **admin panel** to handle user roles, doctor approvals, and system-wide appointments.
- Ensure **secure authentication** with JSON Web Tokens (JWT).
- Store and manage data using **MongoDB**, supporting scalable and flexible schema design.

2.3 Key Features

For Patients (User Role: **user**)

- **User Registration & Login**
 - Register with name, email, password, phone, and role type.
 - Secure login using JWT-based authentication.
- **Doctor Discovery**
 - View a list of approved doctors with details like specialization, experience, consultation fee, timings, and location.
- **Appointment Request**
 - Select a doctor and book an appointment by choosing date and time.
 - Upload supporting documents (e.g., reports, prescriptions) as images.
- **View Appointment Status**
 - View list of all personal appointments with doctor name, date, and status (**pending**, **approved**, etc.).
- **Notifications**
 - See notifications when appointments are approved, rejected, or when status changes.

For Doctors (User with **isdoctor = true**)

- **Doctor Profile Application**
 - Potential doctors apply for a doctor profile with details like specialization, experience, fees, and available timings.
- **Manage Appointment Requests**
 - View appointment requests containing patient info, date/time, and documents.
 - Approve appointments and update status with one click.
- **Download Patient Documents**
 - Download attached documents from patients for diagnosis and review.

For Admin (User Role: **admin**)

- **Admin Dashboard**
 - Overview of all appointments in the system.
- **User Management**
 - View all registered users with their roles and doctor flags.
- **Doctor Profile Management**
 - View all doctor profiles.
 - Approve or reject doctor applications.
- **System-level Notifications**
 - View admin notifications for key events (e.g., new doctor applications).

3. Architecture

The project is developed using the **MERN** stack:

- **Frontend:** React.js
- **Backend:** Node.js with Express.js
- **Database:** MongoDB
- **Communication:** REST APIs with JSON

3.1 Frontend Architecture (React)

The frontend is a **single-page application (SPA)** built with React.

Structure

- Uses **React Router** for client-side routing:
 - **/** – Public landing page (Home)
 - **/login** – Login page

- `/register` – Registration page
- `/userhome` – User dashboard (patients/doctors)
- `/adminhome` – Admin dashboard
- Organized into folders:
 - `/components/admin` – Admin-related components
 - `/components/common` – Shared/public components (Home, Login, Register, Notification)
 - `/components/user` – User dashboard components

State Management

- Uses React's built-in `useState` and `useEffect` hooks for managing local state and side effects.
- Logged-in user data is stored in `localStorage` as `userData`.
- JWT token is stored in `localStorage` as `token`, and included in request headers for protected API calls.

UI Libraries

- **React-Bootstrap**: Tables, Navbars, Cards, Modals, Containers, Buttons.
- **MDB React UI Kit**: Login and registration card layout components.
- **Material UI Icons**: Sidebar icons for navigation (Calendar, Logout, Notifications, etc.).

3.2 Backend Architecture (Node.js + Express.js)

The backend is built with **Express.js** running on Node.js, exposing a set of RESTful APIs under routes such as `/api/user`, `/api/admin`, and `/api/doctor`.

Typical Backend Layers

- **Server Entry (e.g., `server.js` / `index.js`)**
 - Sets up Express app.
 - Connects to MongoDB.
 - Registers route modules (`/api/user`, `/api/admin`, `/api/doctor`).
 - Handles global error handling & middleware setup.
- **Routes**
 - `userRoutes.js`: login, register, get user data, doctor apply, get doctors, user appointments, notifications, etc.
 - `adminRoutes.js`: get all users, get all doctors, approve/reject doctors, get all appointments.
 - `doctorRoutes.js`: get doctor appointments, handle status, document download.
- **Controllers**
 - Each route delegates logic to controller functions that call models and return responses.
- **Middleware**
 - `authMiddleware`:
 - Verifies JWT tokens.
 - Populates `req.user` with decoded user details.
- **Models (MongoDB + Mongoose)**
 - `User` model
 - `Doctor` model
 - `Appointment` model
 - `Notification` model
 - `Document` / file info stored along with appointment

3.3 Database Architecture (MongoDB)

MongoDB is used as a **NoSQL document database**. Collections are designed to store flexible, JSON-like documents.

Key Collections

1. Users Collection (`users`)

Example fields:

- `_id` – Unique user ID
- `fullName` – User's full name
- `email` – Login email (unique)
- `password` – Hashed password
- `phone` – Contact number
- `type` – Role (`admin` or `user`)
- `isdoctor` – Boolean indicating if user also has doctor role
- `notification` – Array of unread notification objects
- `seennotification` – Array of read notifications

2. Doctors Collection (`doctors`)

- `_id`
- `userId` – Reference to `users._id`
- `fullName`
- `email`
- `phone`
- `address`
- `specialization`
- `experience`

- `fees`
- `timings` – Array or object storing start & end time
- `status` – `pending`, `approved`, `rejected`

3. **Appointments Collection** (`appointments`)

- `_id`
- `userId` – Patient user ID
- `doctorId` – Doctor user/doctor ID
- `userInfo` – Embedded basic user data (name, phone, etc.)
- `doctorInfo` – Embedded basic doctor data (name, specialization)
- `date` – Appointment date/time string
- `status` – `pending`, `approved`, `rejected`
- `document` – Info about uploaded file (path, filename)

4. **Notifications (Embedded inside User)**

- Each notification object:
 - `type` – e.g., `appointment-status`
 - `message` – Notification text
 - `onClickPath` – Frontend path to navigate on click
- Stored in `notification` (unread) and `seennotification` (read) arrays in user document.

Relationships

- One **User** can have **one Doctor** profile (if `isdoctor` is true).
- One **Doctor** can have multiple **Appointments**.
- One **User (patient)** can have multiple **Appointments**.

4. Setup Instructions

4.1 Prerequisites

- **Node.js** (v14+ recommended)
- **npm** or **yarn** (Node package manager)
- **MongoDB** Community Server or access to a MongoDB Atlas cluster
- **Git** (optional, for cloning the repository)

4.2 Installation & Environment Setup

Assume the project has two main folders:

- `client` – React frontend
- `server` – Node.js + Express backend

Step 1: Clone the Repository

```
git clone https://github.com/your-username/caresync-portal.git
cd caresync-portal
```

Step 2: Install Frontend Dependencies

```
cd client
npm install
```

This installs React, React Router, Bootstrap, Ant Design, MDB React UI, Axios, and other frontend packages.

Step 3: Install Backend Dependencies

```
cd ../server
npm install
```

This installs Express, Mongoose, JWT, Multer (for file uploads), and other backend libraries.

Step 4: Configure Environment Variables

Create a `.env` file inside the `server` directory with entries like:

```
PORT=8001
```

```
MONGO_URL=mongodb://localhost:27017/caresync
```

```
JWT_SECRET=your_jwt_secret_key_here
```

```
CLIENT_URL=http://localhost:3000
```

- `PORT`: Port on which the backend server will run.
- `MONGO_URL`: Connection string to MongoDB.
- `JWT_SECRET`: Secret key for signing JWT tokens.

5. Folder Structure

5.1 Client (React Frontend)

Example structure:

```
client/
```

```
  src/
```

```
    App.jsx
```

```
    App.css
```

```
    index.jsx
```

```
  components/
```

```
    admin/
```

```
      AdminHome.jsx
```

```
      AdminUsers.jsx
```

```
      AdminDoctors.jsx
```

```
AdminAppointments.jsx
common/
  Home.jsx
  Login.jsx
  Register.jsx
  Notification.jsx
user/
  UserHome.jsx
  ApplyDoctor.jsx
  DoctorList.jsx
  UserAppointments.jsx
images/
  p2.png
  p3.webp
  photo1.png
```

Explanation

- **App.jsx**
 - Defines routes using `react-router-dom`.
 - Wraps all pages and includes footer.
- **App.css**
 - Contains global styles and dashboard layout styling (sidebar, header, etc.).
- **index.jsx**
 - React entry point that renders `<App />` into the DOM.

- **components/admin**
 - `AdminHome.jsx`: Dashboard layout for admin with sidebar and header.
 - `AdminUsers.jsx`: Table showing all users in the system.
 - `AdminDoctors.jsx`: Table of doctor profiles with approve/reject logic.
 - `AdminAppointments.jsx`: Table showing all appointments across users.
- **components/common**
 - `Home.jsx`: Public landing page describing the system.
 - `Login.jsx`: User login page.
 - `Register.jsx`: User registration page.
 - `Notification.jsx`: Tab-based view for unread and read notifications.
- **components/user**
 - `UserHome.jsx`: Main dashboard for patients/doctors with sidebar navigation.
 - `ApplyDoctor.jsx`: Form for users to apply to become doctors.
 - `DoctorList.jsx`: Card-based list of doctors with “Request appointment” modal.
 - `UserAppointments.jsx`: Table showing appointments (different view for doctor and patient).

5.2 Server (Node.js Backend)

A typical backend structure:

`server/`

`server.js`

`package.json`

`.env`

`config/`

```
db.js
models/
  User.js
  Doctor.js
  Appointment.js
routes/
  userRoutes.js
  adminRoutes.js
  doctorRoutes.js
controllers/
  userController.js
  adminController.js
  doctorController.js
middleware/
  authMiddleware.js
uploads/
  documents/    (for uploaded files)
```

Explanation

- **server.js**
 - Initializes Express app.
 - Connects to MongoDB via `db.js`.
 - Uses middleware (`authMiddleware`, body parser, CORS).
 - Mounts route files.

- **config/db.js**
 - Responsible for connecting to MongoDB using Mongoose.
- **models/*.js**
 - Define Mongoose schemas and models for User, Doctor, Appointment.
- **routes/*.js**
 - **userRoutes.js**: Contains routes like login, register, apply doctor, get doctors, user appointments, notifications.
 - **adminRoutes.js**: Contains routes like get all users, get all doctors, approve/reject, get all appointments.
 - **doctorRoutes.js**: Contains routes like get doctor-specific appointments, change appointment status, document download.
- **controllers/*.js**
 - Each controller file contains business logic for corresponding routes.
- **middleware/authMiddleware.js**
 - Validates JWT tokens for protected endpoints.
- **uploads/documents**
 - Stores user-uploaded medical documents for appointments.

6. Running the Application

6.1 Start Backend Server

From the **server** folder:

```
cd server
```

```
npm start
```

- Backend will typically run on **<http://localhost:8001>**.

6.2 Start Frontend Server

From the `client` folder:

```
cd client
```

```
npm start
```

- React app will run on `http://localhost:3000`.

Ensure the backend is running first so API calls work correctly from the frontend.

7. API Documentation

Below are the main APIs used by the frontend. (You can expand with sample JSON in Word.)

7.1 User APIs (`/api/user`)

1. Register User

- **URL:** `POST /api/user/register`
- **Description:** Register a new user (patient/admin).
- **Body Parameters:**
 - `fullName, email, password, phone, type`
- **Response:**
 - `success, message`

2. Login User

- **URL:** `POST /api/user/login`
- **Description:** Authenticate user and return JWT token.
- **Body:**
 - `email, password`

- **Response:**
 - `success, token, userData`

3. Get User Data

- **URL:** `POST /api/user/getuserdata`
- **Headers:**
 - `Authorization: Bearer <token>`
- **Description:** Fetch logged-in user details.

4. Apply for Doctor Profile

- **URL:** `POST /api/user/registerdoc`
- **Headers:** `Authorization`
- **Body:** `{ doctor: {fullName, email, phone, address, specialization, experience, fees, timings}, userId }`

5. Get All Doctors for User

- **URL:** `GET /api/user/getalldoctorsu`
- **Headers:** `Authorization`
- **Description:** Returns list of approved doctors for display on user dashboard.

6. Create Appointment Request

- **URL:** `POST /api/user/getappointment`
- **Headers:** `Authorization, 'Content-Type': 'multipart/form-data'`
- **Body:** `image, date, userId, doctorId, userInfo, doctorInfo`

7. Get User Appointments

- **URL:** `GET /api/user/getuserappointments`
- **Headers:** `Authorization`

- **Query Params:** `userId`

8. Get All Notifications

- **URL:** `POST /api/user/getallnotification`
- **Headers:** `Authorization`
- **Body:** `{ userId }`
- **Description:** Moves unread notifications into seen notifications.

9. Delete All Notifications

- **URL:** `POST /api/user/deleteallnotification`
- **Headers:** `Authorization`
- **Body:** `{ userId }`

7.2 Admin APIs (`/api/admin`)

1. Get All Users

- **URL:** `GET /api/admin/getallusers`
- **Headers:** `Authorization`

2. Get All Doctors

- **URL:** `GET /api/admin/getalldoctors`
- **Headers:** `Authorization`

3. Approve Doctor

- **URL:** `POST /api/admin/getapprove`
- **Headers:** `Authorization`
- **Body:** `{ doctorId, status: 'approved', userid }`

4. Reject Doctor

- **URL:** `POST /api/admin/getreject`

- **Headers:** `Authorization`
- **Body:** `{ doctorId, status: 'rejected', userid }`

5. Get All Appointments (Admin)

- **URL:** `GET /api/admin/getallAppointmentsAdmin`
- **Headers:** `Authorization`

7.3 Doctor APIs (`/api/doctor`)

1. Get Doctor Appointments

- **URL:** `GET /api/doctor/getdoctorappointments`
- **Headers:** `Authorization`
- **Query Params:** `userId`

2. Handle Status (Approve Appointment)

- **URL:** `POST /api/doctor/handlestatus`
- **Headers:** `Authorization`
- **Body:** `{ userid, appointmentId, status }`

3. Download Document

- **URL:** `GET /api/doctor/getdocumentdownload`
- **Headers:** `Authorization`
- **Query:** `appointId`

8. Authentication

8.1 Method

The application uses **JWT (JSON Web Token)** based authentication.

Login Flow

1. User submits email and password via `/api/user/login`.
2. Backend validates credentials and, if valid, signs a JWT with a secret key.
3. JWT is returned to the frontend and stored in `localStorage` as `token`.
4. For protected routes, frontend sends `Authorization: Bearer <token>` in the header.
5. Backend middleware verifies token and attaches user info to `req.user`.

8.2 Authorization & Roles

- **Admin** (`type = 'admin'`)
 - Can access admin APIs and admin dashboard.
- **User** (`type = 'user'`)
 - Default patient role.
- **Doctor** (`isdoctor = true`)
 - User who has an approved doctor profile.
 - Sees additional doctor views (doctor appointment list, approve button).

Protected routes on the backend ensure that only users with valid tokens can access sensitive data.

9. User Interface

9.1 Overview

The UI is designed to be **simple, clean, and responsive**, using a combination of:

- React-Bootstrap for layout components like Navbar, Cards, Tables, and Modals.
- Ant Design for notifications, badges, and tabs.
- Custom CSS for the dashboard layout (sidebar, header, body).

9.2 User Interface Pages Overview

9.2.1 Landing Page (Home.jsx)

The landing page acts as the public entry point to the application.

- It displays a **hero section** introducing CareSync Portal as an online appointment management system.
- A short tagline explains that users can manage clinic visits with less waiting and more control.
- The left side contains descriptive text and call-to-action buttons such as “**Book an appointment**” and “**Join as a patient or doctor**”.
- The right side shows an illustration related to healthcare or scheduling, giving a modern and friendly look.
- Below the hero section, there is an “**About**” section that briefly explains how the portal helps patients, doctors, and admins coordinate appointments.

9.2.2 Login Page

The login page provides a simple and focused interface for existing users.

- At the top, a navigation bar shows the application name and links like **Overview**, **Sign in**, and **Create account**.
- The main content is a card-style layout where one side shows an illustration and the other side shows the login form.

- The form contains:
 - An **email field** for the registered email address.
 - A **password field** for secure sign-in.
 - A primary “**Sign in**” button.
- Below the form, there is a small text link guiding users who do not have an account to the registration page.

9.2.3 Register Page

The registration page allows new users to create an account on the portal.

- A top navigation bar is similar to the login page for consistency.
- The main card layout has two sections: one for the registration form and another for an illustration.
- The form collects:
 - **Full name** of the user.
 - **Email address** used as the login ID.
 - **Password** for account security.
 - **Phone number** for contact purposes.
 - **Role selection** using radio buttons, where the user can choose to register as an administrator/clinic staff or as a patient/doctor.
- A button labelled “**Create account**” submits the registration details.
- A short message at the bottom provides a link back to the login page for existing users.

9.2.4 User Dashboard (UserHome.jsx)

The user dashboard is displayed after a successful login by a normal user (patient or doctor).

- The layout is divided into two main sections: a **sidebar** on the left and a **content area** on the right.
- The **sidebar** contains:
 - The application name and a short description.

- Menu options such as “**My appointments**” and “**Apply as doctor**” (the “Apply as doctor” option is visible only if the user is not already a doctor).
- A **sign out** option at the bottom.
- The **header** in the content area contains:
 - A **notification bell icon** with a count indicating the number of unread notifications.
 - The user’s name, and “Dr.” prefix if the user is also a doctor.
- The **main body** of the dashboard changes based on the selected menu item, showing appointments, doctor application form, notifications, or the home view with doctor listings.

9.2.5 Doctor List View (within UserHome.jsx)

The doctor list view is shown to patients on the dashboard home section.

- Doctors are displayed as **cards** arranged in a grid layout.
- Each card normally shows:
 - Doctor’s **name**.
 - **Specialization** (e.g., General Physician, Cardiologist).
 - **Phone number**.
 - **Clinic address**.
 - **Years of experience**.
 - **Consultation fee**.
 - Available **timings**.
- At the bottom of each card, there is a “**Request appointment**” button that opens a modal to submit an appointment request for that specific doctor.

9.2.6 Appointment Request Modal

When a user clicks on “Request appointment” for a doctor, a modal dialog is displayed.

- The modal shows summary information about the selected doctor, including name and specialization.

- It contains a form with:
 - A **date and time picker** where the patient specifies the preferred appointment date and time.
 - A **file upload field** that allows the user to attach relevant documents such as medical reports or prescriptions.
- The user can either cancel the request or confirm it using a **submit button**, which sends the appointment request to the backend for processing.

9.2.7 Doctor Dashboard View (UserAppointments as Doctor)

When the logged-in user is a doctor (approved doctor profile), the appointments page shows a doctor-specific view.

- The page displays a **table of appointment requests** made by patients for that doctor.
- For each appointment, the table includes:
 - Patient's **name**.
 - **Scheduled time** of the appointment.
 - Patient's **phone number**.
 - A link or button indicating an **attached document**, if available.
 - The **status** of the appointment (such as pending or approved).
- There is an “**Approve**” button or corresponding action that allows the doctor to approve a pending appointment.
- If a document is attached, the doctor can click a **download link/button** to retrieve the file for review.

9.2.8 Admin Dashboard (AdminHome.jsx)

The admin dashboard provides a centralized control panel for administrative users.

- Similar to the user dashboard, it uses a **sidebar + content** layout.
- The **sidebar menu** includes options such as:
 - “**Appointments overview**” – to view all appointments in the system.
 - “**User directory**” – to list all registered users.

- **“Doctor approvals”** – to manage doctor profiles.
- The top header shows the admin user’s name and a notification bell with any pending notifications.
- Based on the selected menu item, the body displays tables and information relevant to appointments, users, or doctors.

9.2.9 Admin – Doctor Management Screen

This screen is accessed via the **“Doctor approvals”** option on the admin sidebar.

- It presents a **table listing all doctor profiles** in the system.
- Each row typically shows:
 - A **profile ID**.
 - Doctor’s **name**.
 - **Email**.
 - **Phone number**.
 - Current **review status** (pending, approved, rejected).
- For doctors in a pending or certain states, the admin can use **Approve** or **Reject** buttons in the **Action** column to update their profile status.
- After an action is performed, the table refreshes to show the latest status, and a confirmation message is shown to the admin.

10. Testing

10.1 Testing Strategy

The project used mainly **manual testing** with some basic automated checks where feasible.

Types of Testing

1. Unit Testing (Logic-level)

- Focused on backend controllers and route handlers.
- Tested login, registration, doctor approval, and appointment creation logic.

2. Integration Testing

- Verified that frontend API calls and backend endpoints work correctly together.
- Example: when a user creates an appointment from the UI, an appointment document is created in MongoDB, and admin/doctor views are updated.

3. Functional Testing

- Validated each feature:
 - Registration
 - Login
 - Doctor application
 - Appointment booking
 - Notifications
 - Admin approvals

4. UI/UX Testing

- Checked responsiveness of the layout on different screen sizes.
- Ensured correct error messages are displayed on invalid inputs.

10.2 Tools Used

- **Postman** – for testing API endpoints.
- **MongoDB Compass** – for inspecting data in MongoDB.
- **Browser DevTools** – for debugging frontend issues.

11. Demo

- A link to a **demo video** hosted on Google Drive..

Link:

[doctor_booking.mp4](#)

12. Known Issues

Some limitations and known issues in the current version:

1. Validation

- Basic validation is present, but more robust checks (strong password, email format) can be enhanced.

2. File Type Restriction

- Currently supports basic image uploads; PDF upload validation may need to be enhanced depending on backend implementation.

3. Mobile UI Optimization

- The layout is responsive, but some complex tables may require further optimization for very small screens.

4. Rate Limiting & Security

- No advanced rate-limiting implemented for API calls, so protection against brute-force login attempts can be improved.

5. Error Handling

- Frontend shows generic “Something went wrong” messages in some cases; more specific user-friendly error feedback can be added.

13. Future Enhancements

Potential improvements and features that can be added in future versions:

1. Role-based Dashboard Analytics

- Admin analytics dashboard: number of active doctors, upcoming appointments, daily/weekly stats.

2. Email / SMS Notifications

- Sending email or SMS reminders to patients and doctors for upcoming appointments.

3. Rescheduling & Cancellation

- Allow patients to modify or cancel appointments and notify doctors automatically.

4. In-app Chat / Messaging

- Secure chat between patient and doctor for pre-consultation questions.

5. Payment Integration

- Online payment for consultation fees using payment gateways like Razorpay/Stripe.

6. Multi-Clinic / Multi-Branch Support

- Support multiple clinics, each with its own doctors and schedules.

7. Advanced Role Management

- Additional roles like receptionist or clinic manager with limited access.

8. Logging & Monitoring

- Adding server-side logging tools and monitoring for better maintenance.